

Sonic Code: un tutor socrático multimodal corporizado para el desarrollo de lógica de programación en entornos STEM

Gabriel Isaías Fallas López
Escuela de Ciencias de la Computación e Informática
Universidad de Costa Rica
San José, Costa Rica
gabrielfallas0803@gmail.com

Resumen—La integración masiva de modelos de lenguaje (LLMs) en la educación en computación ha generado una paradoja pedagógica: el acceso inmediato a soluciones de código erosiona el desarrollo del razonamiento lógico y la descomposición de problemas. Este artículo presenta *Sonic Code*, un tutor socrático que guía al estudiante mediante preguntas reflexivas y andamiaje graduado (Zona de Desarrollo Próximo) sin entregar código directo. El sistema se implementa sobre un LLM local (Ollama + Gemma 3 12B) y se evalúa mediante un estudio piloto de dos condiciones: una condición corporizada (A) con avatar 2D reactivo (Kaplay.js), síntesis de voz neuronal (Piper TTS), reconocimiento de voz local (Whisper STT) y gamificación por anillos; y una condición de control (B) de chat de texto plano con idéntico tutor socrático. Mediante tareas de depuración en Python verificadas por pruebas ocultas, se miden la eficacia autónoma, la carga cognitiva (NASA-TLX), la percepción del agente (Godspeed), la usabilidad (SUS), el afecto post-sesión (PANAS-SF) y un conjunto de métricas conductuales automáticas (latencia, turnos, tiempo de reflexión, uso de voz). En un estudio piloto de $N = 10$ participantes (6 entre-sujetos y 4 intra-sujeto, que aportan 14 observaciones, 7 por condición), la condición corporizada obtuvo puntuaciones de percepción del agente marcadamente superiores (Godspeed global $d = 1.42$; subescala de agrado $d = 1.66$, $p = 0.011$) y una carga cognitiva autopercebida menor (NASA-TLX $d = -0.83$), con usabilidad (SUS) ligeramente mejor y eficacia de resolución equivalente; el *time-to-first-token* se mantuvo bajo 1.5 s en el 92 % de las respuestas. El análisis cualitativo refuerza el patrón: varios participantes —incluidos quienes experimentaron ambas condiciones— prefirieron explícitamente el avatar sobre el texto. Dado el tamaño muestral, los resultados son preliminares y se reportan como tamaños de efecto. Los hallazgos contribuyen a comprender el rol del *embodiment* en tutores socráticos para programación y su efecto sobre la autonomía del aprendiz.

Index Terms—tutoría socrática, agentes virtuales corporizados, modelos de lenguaje, andamiaje cognitivo, educación en programación, gamificación.

I. INTRODUCCIÓN

La educación en ciencias, tecnología, ingeniería y matemáticas (STEM) atraviesa una transformación impulsada por la IA generativa. En computación, la disponibilidad inmediata de soluciones de código produce una *descarga cognitiva*: el estudiante delega la resolución de errores lógicos a un sistema que devuelve el código corregido, eliminando el proceso de prueba y error esencial para formar modelos mentales robustos [1],

[2]. El método socrático —no proporcionar la respuesta, sino guiar mediante preguntas que revelan contradicciones y lagunas— es un antídoto reconocido, pero su escalabilidad es nula en cursos con cientos de estudiantes.

Este trabajo investiga si un tutor socrático basado en LLM, dotado de *corporización* multimodal (avatar reactivo, voz neuronal, reconocimiento de voz y gamificación), mejora la percepción del agente y la eficacia pedagógica autónoma frente al mismo tutor en modo de texto plano. La contribución es triple: (i) un sistema reproducible, completamente local y sin dependencias de nube; (ii) un diseño experimental que aísla el *embodiment* manteniendo constante el texto del tutor; y (iii) un conjunto de métricas conductuales automáticas que reduce la dependencia del auto-reporte.

El resto del artículo se organiza así: la Sección II revisa los trabajos relacionados; la Sección III describe el desarrollo del agente; la Sección IV detalla la metodología; las Secciones V y VI presentan y discuten los resultados; y la Sección VII concluye.

II. TRABAJOS RELACIONADOS

II-A. IA generativa y descarga cognitiva en programación

La irrupción de asistentes de código basados en LLM transformó la práctica de aprender a programar. Phung et al. [1] documentan tanto el potencial de estos modelos para generar retroalimentación como su capacidad de resolver directamente los ejercicios, y Raihan et al. [2] sintetizan su uso en educación en computación. El riesgo pedagógico es la *descarga cognitiva*: si el sistema devuelve la solución, el estudiante omite el ciclo de prueba, error y depuración del que dependen los modelos mentales robustos. De ahí el interés por reorientar estos modelos de “solucionadores” a “tutores” que retienen la respuesta.

II-B. Tutoría socrática con modelos de lenguaje

El método socrático —guiar mediante preguntas en lugar de dar la respuesta— se ha vuelto un patrón recurrente para esa reorientación. *SocraticLM* [3] y *SocraticAI* [4] estructuran la interacción en niveles de andamiaje progresivo; *MathDial* [5]

aporta un corpus de diálogos tutoriales con propiedades pedagógicas ricas que sirve de referencia para entrenar y evaluar; y Jurenka et al. [6] alinean el modelo con objetivos pedagógicos para sostener la indagación sin revelar la solución. En el plano de la evaluación, Tack y Piech [7] proponen criterios para juzgar la calidad tutorial de un agente, un problema no trivial dado que “parecer” un buen tutor no implica producir aprendizaje. Estudios más recientes reportan ganancias de aprendizaje, compromiso metacognitivo y mejor percepción del andamiaje en cursos STEM introductorios [8], [9], [10], y Essel et al. [11] vinculan específicamente el cuestionamiento mediado por IA con el compromiso del estudiante —trabajo del que adaptamos nuestra escala de apoyo pedagógico. Una limitación transversal es que la *planificación adaptativa* del diálogo (decidir cuándo y cuánto andamiar) sigue siendo incipiente [12]; nuestro sistema adopta una política simple y explícita de tres niveles ligada al estancamiento del estudiante.

II-C. Agentes conversacionales corporizados en educación

Más allá del texto, una línea de trabajo dota al tutor de un cuerpo —avatar, voz, gestos. La revisión de alcance de Sonlu et al. [13] sintetiza una década de agentes conversacionales corporizados y concluye que la *presencia social* suele mejorar la experiencia, pero con resultados heterogéneos según la tarea, la población y el grado de realismo. En programación y entornos inmersivos, propuestas multimodales y de realidad virtual [14], [15] sugieren que la corporización puede favorecer el aprendizaje y la motivación, aunque advierten dos riesgos: una carga perceptiva adicional y el rechazo cuando el realismo es casi-humano pero imperfecto (*Valle Inquietante*) [16]. Nuestro uso de un personaje estilizado y reconocible (un sprite 2D) en lugar de un humano fotorrealista es, en parte, una respuesta a este último riesgo.

II-D. Carga cognitiva, novedad y medición

La evidencia agregada invita a la cautela. El meta-análisis de Xu et al. [17] estima un efecto *positivo pero pequeño* de los agentes pedagógicos sobre el aprendizaje, y el de Li et al. [18] muestra que solo reducen levemente la carga cognitiva e incluso la *aumentan* en materiales complejos o sesiones largas —un riesgo directamente pertinente para una tarea densa como la depuración. A ello se suma que las medidas de auto-reporte y las conductuales correlacionan débilmente [19], por lo que apoyarse solo en cuestionarios puede inducir conclusiones frágiles, especialmente con muestras pequeñas. Estos hallazgos motivan nuestras decisiones metodológicas: medir la carga (NASA-TLX) y el afecto (PANAS-SF) con instrumentos validados, complementarlos con telemetría conductual objetiva, y tratar el atractivo inicial del personaje como un posible *efecto de novedad* a controlar.

II-E. Brecha y posicionamiento

En conjunto, la literatura aborda por separado (a) la calidad socrática del diálogo y (b) el efecto de la corporización, y con frecuencia compara condiciones cuyo *contenido* de tutor difiere, lo que confunde el efecto del *embodiment* con el de una

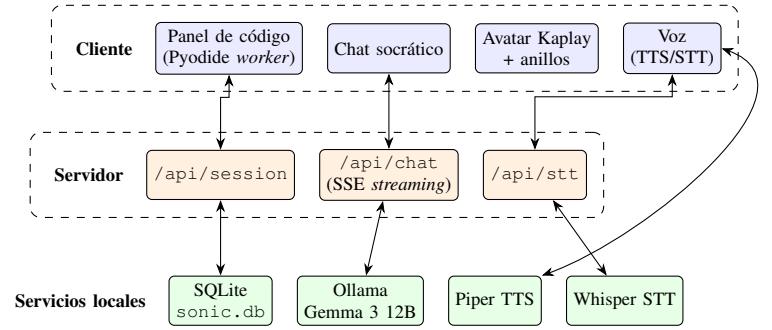


Figura 1. Arquitectura en tres capas. El texto del tutor es idéntico en ambas condiciones; el avatar, la voz y la gamificación (sombreado) solo se activan en la Condición A.

persona o un guion distinto. Además, la eficacia suele medirse por auto-reporte. Este trabajo se posiciona en esa brecha en tres puntos: (i) compara dos condiciones con *texto de tutor idéntico*, aislando el *embodiment* (avatar + voz + gamificación); (ii) mide la eficacia autónoma con verificación *objetiva* (pruebas ocultas) en lugar de auto-reporte; y (iii) combina auto-reporte, telemetría conductual y análisis cualitativo para sortear la débil correlación entre tipos de medida.

III. DESARROLLO DEL AGENTE

III-A. Arquitectura del sistema

El sistema es una aplicación web **Next.js 15** (App Router, React 19, TypeScript) cuyos servicios de IA corren *en local*, sin claves de API ni dependencias de nube (Fig. 1). Se organiza en tres capas: (i) un **cliente** en el navegador con el panel de código, el chat, el avatar y la voz; (ii) un **servidor** Next.js que expone rutas de API y orquesta la inferencia; y (iii) tres **servicios locales** —el modelo de lenguaje, la síntesis y el reconocimiento de voz— más una base de datos embebida para la telemetría.

Los componentes principales son:

- **Motor de razonamiento.** Ollama sirve a Gemma 3 12B (cuantización Q4_K_M, inferencia en GPU local). La ruta `/api/chat` construye el *system prompt* socrático, envía el historial al modelo y retransmite la respuesta al cliente por *streaming* de tokens (Server-Sent Events) para minimizar la latencia percibida (objetivo < 1,5 s de *time-to-first-token*).
- **Avatar 2D (Condición A).** Un *canvas* Kaplay.js renderiza un sprite reactivo con **ocho estados expresivos**; su control se detalla en la Sección III-B.
- **Voz (Condición A).** Síntesis neuronal Piper TTS (español) con respaldo a la Web Speech API; reconocimiento de voz local Whisper STT vía `/api/stt`.
- **Gamificación (Condición A).** Sistema de anillos (recompensa al acertar), mini-juego de transición entre tareas y efectos de sonido/música por zona temática.
- **Ejecución de código.** Un editor real ejecuta Python en el navegador (Pyodide en un *Web Worker* con *timeout*) y anexa un arnés de pruebas ocultas que determina objetivamente la resolución autónoma.

- **Telemetría.** Persistencia en una base SQLite embebida; exportación a CSV/JSON/transcripciones y panel del facilitador (Sección III-D).

III-B. Andamiaje socrático y control afectivo del avatar

El comportamiento pedagógico se especifica en un *system prompt* compartido por ambas condiciones, con tres reglas duras: no generar ni corregir código, no señalar la línea errónea, y terminar siempre con una pregunta reflexiva (respuestas de 3–4 oraciones, sin emojis). El conocimiento del error real se entrega al modelo solo para razonar, nunca para revelarlo.

III-B0a. Tres niveles de andamiaje (ZPD): Operacionalizando la Zona de Desarrollo Próximo [20], la ayuda escala *solo* si el estudiante acumula tres o más turnos sin avanzar:

- **Nivel 1 (turnos 1–2): preguntas amplias** sobre el objetivo del código (p. ej. “¿qué condición debe volverse falsa para que el bucle se detenga?”).
- **Nivel 2 (turnos 3–4): orientación conceptual** hacia la estructura o el invariante (p. ej. “¿qué valor tiene esa variable en la primera iteración frente a la segunda?”).
- **Nivel 3 (turnos 5+): pista estructural, todavía sin código**, dirigiendo la atención a la línea o a la complejidad (“si la lista tiene N elementos, ¿cuántas comparaciones hace tu algoritmo?”).

III-B0b. Detección de estado afectivo: El sistema combina dos mecanismos. Primero, una *regla de des-escalada*: ante marcadores léxicos de frustración (“no entiendo”, “no sé”, “me rindo”, “ayuda”, “no puedo”) el tutor valida el esfuerzo, retrocede un nivel de andamiaje y pide reformular el problema en palabras simples. Segundo, una *inferencia afectiva* que el propio modelo realiza sobre el turno del estudiante para seleccionar el estado expresivo del avatar. Estos estados se transmiten mediante una etiqueta [AVATAR_STATE: estado] que el modelo añade al final de la respuesta (Condición A) y que el cliente *elimina* antes de mostrar el texto, de modo que la transcripción visible es idéntica a la de la Condición B. Los ocho estados son: cuatro *inferidos por el modelo* según el contenido —**curious** (pregunta reflexiva, por defecto), **happy** (el estudiante se acerca a la solución), **empathetic** (frustración) y **encouraging** (motivar a seguir)— y cuatro *dirigidos por eventos de interacción* en el cliente —**thinking** (mientras se genera la respuesta), **speaking** (durante la locución TTS), **listening** (durante la captura de voz/espera) e **idle** (reposo). Así, el avatar reacciona tanto al *qué* dice el estudiante como al *momento* de la interacción.

III-C. Decisión de diseño clave: aislar el embodiment

Ambas condiciones comparten el *mismo* texto de tutor; la única diferencia a nivel de *prompt* es el bloque de control del avatar añadido en la Condición A (cuya única salida, la etiqueta de estado, se elimina antes de mostrarse). Así, la comparación A vs. B mide el efecto de la corporización (sprite + voz + gamificación) y no una persona o un texto distinto. Las Figuras 2 y 3 muestran ambas interfaces.

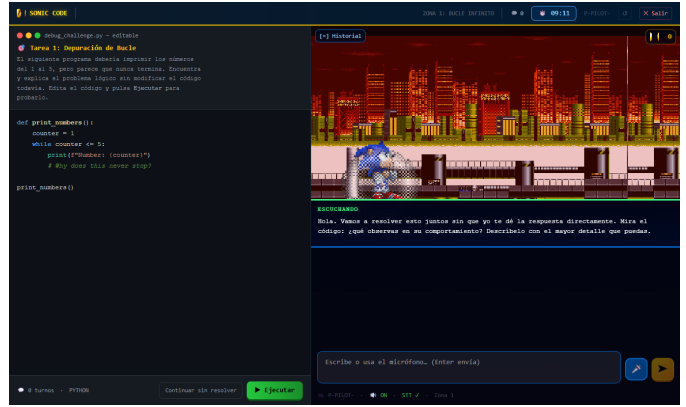


Figura 2. Condición A (corporizada): avatar Sonic en *canvas* Kaplay.js, HUD de anillos y panel de código con ejecución en Pyodide.

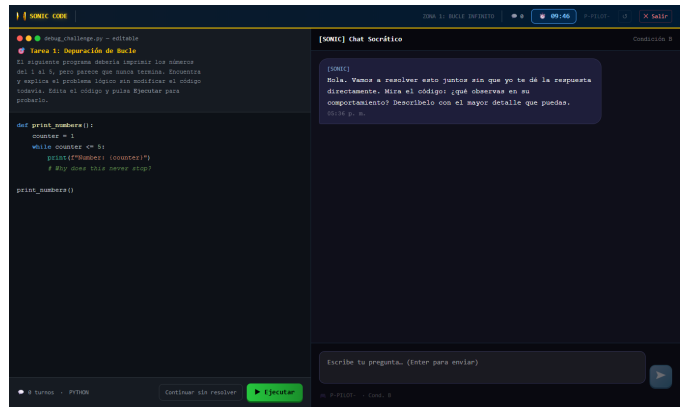


Figura 3. Condición B (control): chat de texto plano con el mismo tutor socrático, sin avatar, voz ni gamificación.

III-D. Telemetría y datos registrados

Toda la sesión se persiste en una base SQLite embebida (*sonic.db*) anclada a la raíz del proyecto. El esquema tiene cuatro tablas: *sessions* (una fila por participante con el registro completo en JSON más columnas indexadas *condition/isPilot/tiempos*), *events* (bitácora append-only de hitos: inicio, resultado de tarea, cuestionario, cierre), *assignment* (contador de contrabalanceo persistente, para que la numeración P-001... y el balance A/B sobrevivan reinicios) y *meta*. Por cada sesión se almacenan, y se usan así:

- **Mensajes:** rol, contenido, marca de tiempo, *inputMode* (voz/teclado) y *latencyMs/ttftMs* por respuesta del tutor. Sirven para la latencia (RQ4), el *think-time* entre turnos (reflexión, RQ2), la verbosidad y el uso de voz (RQ1).
- **Resultados de tarea:** *resolvedAutonomously* y *testsPassed* (eficacia objetiva, RQ2), *turns*, *timeSpentSeconds*, *codeRunAttempts* y *codeEdited* (esfuerzo y proceso), *ringsCollected* (compromiso gamificado) y la *condition* de esa tarea (clave del diseño *crossover*).

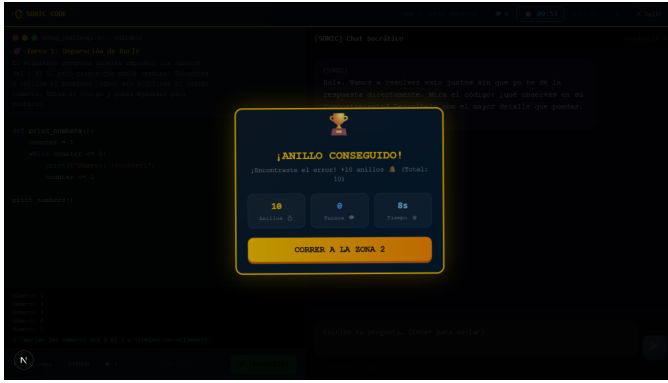


Figura 4. Verificación objetiva de la resolución: el arnés de pruebas ocultas confirma que la corrección del estudiante pasa.

- **Cuestionarios:** respuestas crudas y puntajes derivados por instrumento, etiquetados por condición, que alimentan RQ1 (Godspeed, apoyo pedagógico, SUS) y RQ3 (NASA-TLX, PANAS-SF).
- **Metadatos del diseño:** design, sequence y los tiempos de inicio/cierre, que permiten combinar las observaciones entre-sujetos y crossover.

Tres exportaciones derivan de esta base: un CSV con una fila por participante (métricas y puntajes), un JSON de fidelidad completa y las transcripciones en JSONL para codificación cualitativa.

III-E. Implementación

El cliente y el servidor se construyeron como una única aplicación Next.js con renderizado del lado del cliente para las vistas interactivas. El **chat** mantiene el historial en React y consume `/api/chat`, que abre una conexión SSE: el servidor invoca a Ollama, lee el flujo de tokens y los reenvía conforme llegan, midiendo el *time-to-first-token* en el primer fragmento. El **panel de código** integra un editor y un *Web Worker* que inicializa Pyodide una sola vez; al ejecutar, concatena el código del estudiante con el arnés de pruebas oculto y captura la línea `__TESTS__` que reporta *passed/detail*, con un *timeout* que sirve además para detectar bucles infinitos (Fig. 4). El **avatar** se dibuja en un *canvas* Kaplay.js independiente; la **voz** usa el endpoint `/api/stt` (Whisper) para transcribir y Piper para sintetizar la respuesta. La **asignación** de condición/orden se resuelve en el servidor por bloques permutados y se persiste, evitando que el participante elija condición. Las **tareas** de depuración son: (T1) un bucle `while` que no termina por un contador que no se actualiza; y (T2) un algoritmo de fuerza bruta con bucles anidados que debe optimizarse a $O(n)$ con un conjunto; para el diseño *crossover* existe una segunda variante equivalente de cada una. Todo el sistema se empaqueta con Docker Compose (la aplicación, Ollama y el servicio de voz) para una ejecución reproducible con un solo comando.

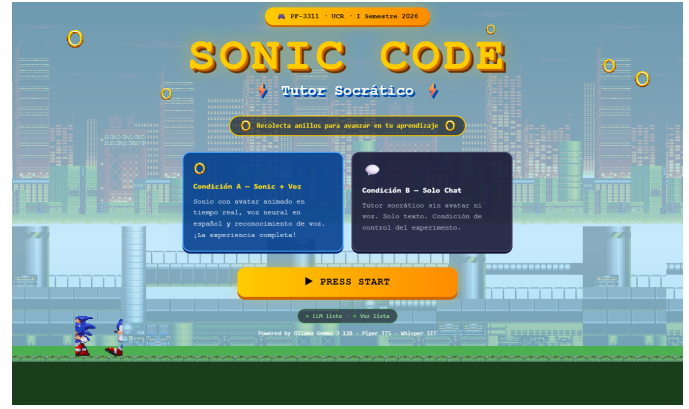


Figura 5. Pantalla de inicio. El participante pulsa *PRESS START* y el servidor asigna la condición/orden; los botones de prueba quedan ocultos durante la recolección real.

IV. METODOLOGÍA

IV-A. Preguntas de investigación

- **RQ1.** ¿Cómo afecta la corporización a la percepción de naturalidad, presencia social y apoyo pedagógico percibido?
- **RQ2.** ¿En qué medida la modalidad influye en la eficacia pedagógica autónoma (resolución sin código directo, turnos, tiempo)?
- **RQ3.** ¿Qué relación existe entre la modalidad y la carga cognitiva (NASA-TLX) y el afecto post-sesión (PANAS-SF)?
- **RQ4.** ¿Se mantiene la latencia multimodal por debajo de 1.5 s de forma consistente?

IV-B. Diseño del estudio

Este artículo reporta un *estudio piloto de viabilidad* ejecutado en dos fases con un diseño mixto. La **Fase 1** (6 participantes) empleó un diseño **entre-sujetos**: cada participante se asignó a una condición (A corporizada o B texto) y resolvió en ella las dos tareas de depuración. La **Fase 2** (4 participantes) empleó un diseño **intra-sujeto (crossover)**, en el que cada participante resolvió ambas tareas en *ambas* condiciones. En ambas fases la asignación (de condición o de orden) la realiza el servidor mediante *aleatorización por bloques permutados*, de modo que el participante nunca elige su condición y el balance A/B queda garantizado por bloque. Se adopta un esquema *post-only*: **no se administran formularios al inicio** —el participante entra directo a la experiencia— lo que reduce la fricción inicial y evita el *priming* de medidas previas. Como cada sesión *crossover* aporta una observación por condición y cada sesión entre-sujetos aporta una, ambos diseños se combinan en el contraste A vs. B por condición (14 observaciones; 7 por condición).

IV-B0a. Diseño crossover con variantes de tarea:

En la Fase 2, cada participante realiza cuatro ejercicios en dos bloques (las dos tareas en una condición y luego en la opuesta), con el *orden* de condiciones contrabalanceado (A→B / B→A). Para evitar que el participante resuelva el

problema idéntico dos veces (efecto de práctica/memoria — señalado por un participante de la Fase 1), cada tarea tiene **dos variantes equivalentes** en dificultad y constructo (dos errores de terminación de bucle distintos; dos algoritmos de fuerza bruta con bucles anidados optimizables a $O(n)$ con un *set*): el Bloque 1 usa la variante 1 y el Bloque 2 la variante 2. Como la variante está ligada a la posición del bloque y el orden de condiciones es contrabalanceado, la dificultad de variante queda balanceada entre A y B. El diseño intra-sujeto controla las diferencias individuales y maximiza la potencia con la N pequeña típica de un estudio de curso; los cuestionarios específicos de cada condición se responden una vez por bloque y se propone como diseño principal para la recolección a escala.

IV-C. Participantes

La muestra piloto fue de $N = 10$ participantes reclutados por conveniencia (6 en la Fase 1 entre-sujetos —3 por condición— y 4 en la Fase 2 *crossover*), con edad media de 24,9 años (DE = 10,3; rango 19–53) y composición de 9 hombres y 1 mujer. La experiencia previa en programación fue marcadamente heterogénea (5 *ninguna*, 2 *principiante*, 3 *avanzada*), al igual que la familiaridad con agentes de IA. Combinando ambas fases, el contraste por condición cuenta con 7 observaciones en A y 7 en B. La homogeneidad de género, el único participante atípico de mayor edad y el reclutamiento por conveniencia se discuten como limitaciones (Sección VI).

IV-D. Instrumentos y métricas

La Tabla I resume los instrumentos. Las medidas de auto-reporte (todas administradas una sola vez al final) se complementan con métricas conductuales capturadas automáticamente del flujo de mensajes, sin interrumpir al participante: latencia (*time-to-first-token* medio y percentil 95), turnos, tiempo por tarea, *think-time* entre turnos, verbosidad (caracteres/palabras), uso de voz vs. teclado y anillos. Esta combinación es deliberada: las medidas de auto-reporte y las conductuales suelen correlacionar débilmente [19], por lo que ante una muestra pequeña se prioriza la evidencia conductual y un análisis cualitativo de las conversaciones (codificación temática) sobre los puntajes de cuestionarios. La Figura 6 muestra un ítem del conjunto de cuestionarios tal como lo responde el participante en la aplicación.

IV-D0a. Verificación de la manipulación: La condición se aplica en el servidor y determina de forma determinista la interfaz: la Condición A habilita el avatar Kaplay, la voz Piper/STT y la gamificación por anillos, mientras que la Condición B las elimina por completo; el *texto* del tutor es idéntico en ambas. Así, cualquier diferencia observada es atribuible al *embodiment* y no al contenido del tutor.

IV-E. Protocolo de interacción

Protocolo ejecutado en el piloto (entre-sujetos): (1) el participante pulsa *PRESS START* y el servidor asigna condición e identificador anónimo; (2) resuelve la Tarea 1 (máx. 10 min)

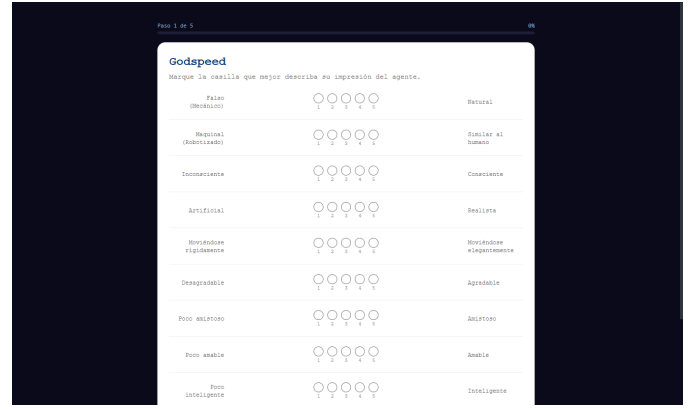


Figura 6. Ejemplo de instrumento (Godspeed) administrado en la aplicación al final de la interacción, con barra de progreso del conjunto de cuestionarios.

Cuadro I
CONJUNTO DE INSTRUMENTOS

Instrumento	Constructo	RQ
Demográficos	Perfil de la muestra	—
Godspeed [21]	Antropomorfismo, agrado	RQ1
Apoyo pedagógico (5 ítems) [11]	Apoyo socrático percibido	RQ1
SUS [22]	Usabilidad	RQ1
NASA-TLX [23]	Carga cognitiva (RTLX)	RQ3
PANAS-SF [24]	Afecto post-sesión	RQ3
Preguntas abiertas	Apreciación cualitativa	—
Telemetría automática	Eficacia, latencia	RQ2, RQ4

interactuando por texto (ambas condiciones) o voz (Condición A); (3) mini-juego de transición; (4) resuelve la Tarea 2 (máx. 10 min); (5) completa los cuestionarios finales. El tutor aplica andamiaje de tres niveles y envía mensajes proactivos tras 60 s de inactividad, sin entregar código. En el protocolo *crossover* refinado, este ciclo de dos tareas se repite en la condición opuesta (con las variantes de tarea) y el conjunto de cuestionarios específico de la condición se responde al final de cada bloque.

IV-F. Recolección de datos

El flujo de recolección (Fig. 7) es totalmente automatizado: desde la asignación en el servidor hasta la persistencia y exportación, sin intervención manual que introduzca errores de transcripción. Cada sesión se guarda en una base de datos SQLite local bajo un identificador anónimo (P-001, ...), sin nombre ni correo. El sistema exporta un CSV con una fila por participante (ver el diccionario de datos del repositorio), un JSON de fidelidad completa y las transcripciones de conversación en JSONL para la codificación cualitativa.

El análisis inferencial (Mann–Whitney U , δ de Cliff, d de Cohen) se ejecuta sobre el CSV con el *script* incluido en el repositorio.

V. RESULTADOS

Se reportan estadísticas descriptivas por condición (media, DE) y, dado el tamaño muestral reducido (7 observaciones por condición), se prioriza el **tamaño de efecto** (d de Cohen;

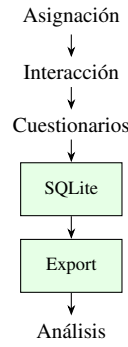


Figura 7. Flujo de recolección y análisis de datos, de extremo a extremo.

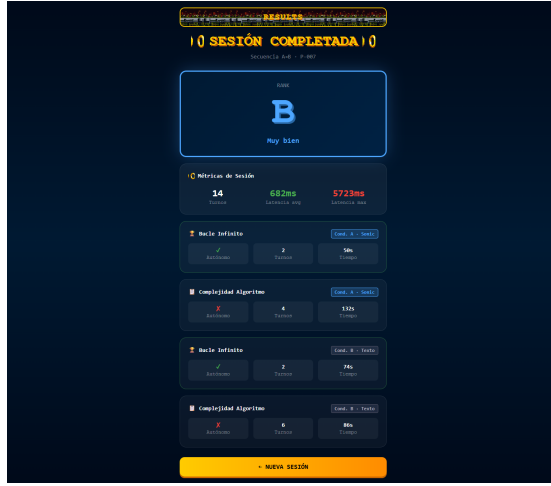


Figura 8. Pantalla de resultados de una sesión *crossover* (P-007, secuencia A→B): rango resumen, métricas de sesión y las cuatro corridas (ambas tareas en cada condición) con su etiqueta de condición.

A–B) sobre la significancia inferencial. La Tabla II resume las métricas principales. La unidad del contraste es la *observación por condición*: cada participante *crossover* aporta una observación a cada condición y cada participante entre-sujetos una a su condición, totalizando 7 en A y 7 en B. La Figura 8 ilustra la pantalla de resultados que cierra cada sesión, con las cuatro corridas etiquetadas por condición en el diseño *crossover*.

V-0a. Percepción del agente (RQ1): La corporización produjo el efecto más grande y consistente: Godspeed global fue muy superior en A (4,24 vs. 2,89; $d = 1,42$; $p = 0,055$), y la subescala de *agrado* alcanzó significancia pese al tamaño muestral (5,00 vs. 3,19; $d = 1,66$; δ de Cliff = 0,71; $p = 0,011$), con antropomorfismo e inteligencia también favorables a A ($d = 1,07$ y 1,18). La usabilidad (SUS) fue solo ligeramente mayor en A ($d = 0,42$) y el apoyo pedagógico percibido fue equivalente ($d = 0,11$), lo que sugiere que el efecto se concentra en la *presencia social* del avatar y no en una interfaz más usable ni en una percepción distinta del andamiaje.

V-0b. Eficacia autónoma (RQ2): La tasa de resolución autónoma fue idéntica entre condiciones (0,29 en ambas; $d = 0$). La Condición A requirió algo menos de turnos por tarea

Cuadro II
RESULTADOS DESCRIPTIVOS POR CONDICIÓN, MEDIA (DE), 7 OBSERVACIONES CADA UNA. d : TAMAÑO DE EFECTO DE COHEN ($A - B$); * $p < 0,05$ (MANN-WHITNEY). EN Turnos, Tiempo Y NASA-TLX UN VALOR MENOR ES MEJOR.

Métrica	Cond. A	Cond. B	d
Resolución autónoma (0–1)	0,29 (0,39)	0,29 (0,39)	0,00
Turnos por tarea	3,4 (1,6)	5,7 (5,1)	–0,60
Tiempo por tarea (s)	230 (143)	338 (369)	–0,39
TTFT mediano (ms)	218	244	—
% latencia < 1,5 s	88	94	—
SUS (0–100)	72,5 (11,9)	65,0 (22,1)	0,42
Apoyo pedagógico (1–5)	3,83 (1,30)	3,69 (1,35)	0,11
NASA-TLX (RTLX)	57,9 (22,0)	73,3 (14,5)	–0,83
Godspeed global (1–5)	4,24 (0,40)	2,89 (1,29)	1,42
<i>antropomorfismo</i>	3,64 (0,84)	2,57 (1,13)	1,07
<i>agrado*</i>	5,00 (0,00)	3,19 (1,54)	1,66
<i>inteligencia</i>	4,36 (0,56)	3,00 (1,53)	1,18
PANAS afecto +	18,4 (2,9)	15,9 (4,0)	0,73
PANAS afecto –	10,9 (4,5)	10,3 (4,1)	0,13

(3,4 vs. 5,7; $d = -0,60$) y de tiempo ($d = -0,39$), pero con esta muestra y la heterogeneidad de experiencia previa no puede atribuirse causalidad al *embodiment*.

V-0c. Carga cognitiva y afecto (RQ3): La carga autopercebida (NASA-TLX) fue *menor* en A (57,9 vs. 73,3; $d = -0,83$) y el afecto positivo (PANAS) mayor ($d = 0,73$), sin diferencia relevante en afecto negativo. Es decir, en este piloto la corporización no aumentó la carga —el riesgo advertido por [18]— sino que acompañó a una experiencia percibida como más ligera y positiva.

V-0d. Latencia (RQ4): Sobre 119 respuestas del tutor, el *time-to-first-token* se mantuvo bajo 1,5 s en el 92 % de los casos (mediana 234 ms). El cumplimiento fue algo menor en la Condición A (88 % vs. 94 %), atribuible a respuestas ocasionalmente más lentas en las sesiones con voz y avatar y en las realizadas de forma remota; aun así, la mediana de A (218 ms) fue equivalente, confirmando la viabilidad del objetivo de latencia con inferencia local.

V-A. Hallazgos cualitativos

El análisis temático de las respuestas abiertas (codificadas según el esquema del repositorio) reveló patrones coherentes con los cuantitativos:

- **Preferencia por la corporización, incluso intra-sujeto.** El testimonio más fuerte proviene de participantes *crossover* que vivieron ambas condiciones: “El chat de texto fue molesto y poco práctico... no veo que el sistema esté interactuando conmigo” y “Prefiero 100 % el usar la opción del avatar de Sonic mientras aprendo algo tan difícil como... programar” (P-007). En la fase entre-sujetos, quienes solo tuvieron texto también pidieron el avatar: “No me gustó no ver a Sonic” (P-005); “Prefiero el aprender junto con un agente virtual como Sonic” (P-003). Ningún participante expresó la preferencia inversa.
- **Presencia social y guía percibida.** Se valoró la reactividad del avatar: “poder charlar con Sonic y ver cómo este reaccionaba ante mis comentarios. Fue una guía para

mi” (P-006); “*es Sonic... lo que lo hace más interactivo*” (P-008); “*es amable, amigable y divertido*” (P-002).

- **El andamiaje socrático se percibió.** Un participante de texto destacó “*que de verdad me fuera ayudando... sin darme la respuesta*” (P-005), evidenciando que la restricción de no entregar código fue percibida como apoyo y no como carencia.
- **Presión de la gamificación.** Un participante reportó “*un poco de presión para contestar rápido, porque sentía que podía perder el nivel*” (P-009), señal de que el sistema de anillos puede añadir demanda temporal —un matiz relevante para la carga cognitiva (RQ3).
- **Fricciones de usabilidad y prerrequisitos.** Se mencionaron el tiempo de respuesta y la falta de explicación inicial de los controles (P-002), y la barrera de conocimiento previo en participantes sin experiencia (“*como no sé nada sobre programación me tomó más tiempo entender el código*”, P-009; similar en P-001).
- **Efecto de repetición.** Un participante de la Fase 1 señaló que repetir el mismo ejercicio no aportaba (“*no me gustó volver a hacer las mismas preguntas*”, P-006), hallazgo que motivó directamente las variantes de tarea introducidas en la Fase 2.

VI. ANÁLISIS Y DISCUSIÓN

Los resultados del piloto, aunque preliminares, dibujan un patrón coherente entre las medidas cuantitativas y cualitativas. Respecto a **RQ1**, la corporización elevó de forma sustancial la percepción del agente (Godspeed global $d = 1,42$; agrado $d = 1,66$ con $p = 0,011$) con solo una mejora menor de usabilidad ($d = 0,42$) y sin diferencia en el apoyo pedagógico percibido ($d = 0,11$): el efecto se concentra en la *presencia social* del avatar y la voz, no en una interfaz más fácil ni en una percepción distinta del andamiaje. Esto concuerda con los efectos de presencia social reportados en agentes corporizados [14], [13] y se ve reforzado por el dato cualitativo más fuerte: participantes que experimentaron *ambas* condiciones prefirieron sin ambigüedad el avatar sobre el texto.

En cuanto a **RQ3**, la condición corporizada se asoció a una carga cognitiva autopercebida *menor* (NASA-TLX, $d = -0,83$) y a un afecto positivo mayor ($d = 0,73$). Es decir, en este contexto no se materializó el riesgo de que el agente *aumente* la carga en tareas densas advertido por [18]; una interpretación plausible es que la voz y el andamiaje visual redujeron la carga *extrínseca* de la interacción. No obstante, un participante reportó sentir presión temporal por el sistema de anillos, lo que sugiere que la gamificación puede introducir demanda adicional y debe calibrarse. Para **RQ2**, la eficacia de resolución fue *idéntica* entre condiciones ($d = 0$): la corporización mejoró la experiencia sin penalizar (ni, en este piloto, potenciar) el desempeño objetivo. Finalmente, **RQ4** se cumplió: 92 % de las respuestas bajo 1.5 s con inferencia totalmente local (mediana 234 ms), con un cumplimiento algo menor en la Condición A por respuestas puntuales más lentas en sesiones con voz y remotas.

Coherente con la débil correlación entre auto-reporte y conducta [19] y con la recomendación de no depender de los cuestionarios ante N pequeño, la evidencia más robusta de este piloto proviene de la *convergencia* entre el efecto de percepción (Godspeed, con agrado significativo), la señal conductual (menos turnos) y la preferencia explícita —incluso intra-sujeto— por la corporización en las respuestas abiertas.

VI-0a. Limitaciones: Las conclusiones son preliminares y deben leerse con cautela. (i) *Tamaño muestral*: con 7 observaciones por condición los intervalos de confianza son amplios; aunque la subescala de agrado alcanzó significancia, la mayoría de los efectos no, y sus magnitudes pueden variar con más datos. (ii) *Diseño mixto y confusión*: parte de la evidencia proviene de comparaciones entre-sujetos donde la experiencia previa no quedó balanceada, por lo que las diferencias de eficacia (RQ2) no son atribuibles limpiamente al *embodiment*; el diseño *crossover* (Fase 2) mitiga esto y se propone como diseño principal a escala. (iii) *Homogeneidad de la muestra*: predominio de hombres jóvenes (9 de 10), lo que limita la generalización. (iv) *Efecto de novedad*: la valoración de la Condición A pudo inflarse por la familiaridad y el atractivo del personaje Sonic, un efecto que suele atenuarse con el uso repetido. (v) *Medición post-only* del afecto (sin línea base) y *dependencia del hardware* y la red para la latencia.

VII. CONCLUSIONES Y TRABAJO FUTURO

Se presentó *Sonic Code*, un tutor socrático multimodal corporizado, completamente local y reproducible, junto con un diseño que aísla el *embodiment* (texto de tutor idéntico) y verifica la autonomía con pruebas ocultas en lugar de auto-reporte. La evidencia del estudio piloto ($N = 10$), aunque preliminar, es internamente coherente: la corporización se asoció a una percepción del agente notablemente mejor (Godspeed global $d = 1,42$; agrado $d = 1,66$, $p = 0,011$) y a una carga cognitiva autopercebida menor (NASA-TLX, $d = -0,83$), con eficacia de resolución equivalente, mientras la latencia local cumplió el objetivo ($< 1,5$ s en el 92 % de las respuestas). El dato cualitativo más claro —que los participantes, incluidos quienes vivieron ambas condiciones, prefirieran el avatar— converge con la señal cuantitativa de presencia social. La principal conclusión, por tanto, es que el *embodiment* parece operar sobre la dimensión socio-afectiva y de carga percibida sin penalizar la eficacia, lo que justifica un estudio confirmatorio con mayor poder. El trabajo futuro incluye la ejecución del diseño *crossover* con variantes de tarea aquí descrito, planificación adaptativa del diálogo [12], más tareas y dominios, y una muestra mayor y más diversa.

VII-0a. Proyección de publicación: Sí se proyecta dar continuidad a este trabajo. Se planea ejecutar el estudio completo con el diseño intra-sujeto (*crossover*) y las variantes de tarea ya implementadas en el sistema, ampliando la muestra y su diversidad, previo sometimiento y aprobación del Comité Ético Científico (CEC) de la Universidad de Costa Rica, con miras a una eventual publicación académica. La presente

entrega constituye el piloto de viabilidad que fundamenta dicha proyección.

REFERENCIAS

- [1] T. Phung *et al.*, “Generative ai for programming education: Benchmarking chatgpt, gpt-4, and human tutors,” *arXiv preprint*, 2023.
- [2] N. Raihan *et al.*, “Large language models in computer science education: A systematic literature review,” *ResearchGate preprint*, 2025.
- [3] Y. Liu *et al.*, “Socraticlm: Exploring socratic personalized teaching with large language models,” *ResearchGate preprint*, 2024.
- [4] K. Sunil and A. Thakkar, “Socraticai: Transforming llms into guided cs tutors through scaffolded interaction,” *arXiv preprint*, 2025.
- [5] J. Macina *et al.*, “Mathdial: A dialogue tutoring dataset with rich pedagogical properties,” *arXiv preprint*, 2023.
- [6] S. Jurenka *et al.*, “From problem-solving to teaching problem-solving: Aligning llms with pedagogy using reinforcement learning,” *arXiv preprint*, 2024.
- [7] A. Tack and C. Piech, “Clean code tutoring: Makings of a foundation,” *ResearchGate preprint*, 2022.
- [8] S. R. Ali *et al.*, “From tool to tutor: Socratic ai tutoring, metacognitive engagement, and prior knowledge as determinants of learning gains in gateway stem courses,” *Regional Lens*, 2026.
- [9] L. Xi *et al.*, “Investigating the effects of an llm-based socratic conversational agent on student learning in higher education,” *NIE Repository*, 2026.
- [10] S.-C. Kao *et al.*, “Socratic ai in k–12 science classrooms,” *ResearchGate preprint*, 2025.
- [11] H. B. Essel *et al.*, “Ai-mediated questioning and engagement,” *Sustainability*, vol. 17, no. 21, p. 9508, 2024.
- [12] Z. Dong *et al.*, “Learning from long-term engagement: Adaptive tutoring dialogue planning for personalized education,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.
- [13] S. Sonlu *et al.*, “A scoping review of embodied conversational agents in education: Trends and innovations from 2014 to 2024,” *ResearchGate preprint*, 2024.
- [14] M. El Hajji *et al.*, “An architecture for intelligent tutoring in virtual reality: Integrating llms and multimodal interaction for immersive learning,” *Information*, vol. 16, no. 7, p. 556, 2025.
- [15] X. Yang and Y. Zhan, “‘embodied’ ai in virtual reality improves programming student learning,” *NC State University News*, 2025.
- [16] M. Mori, “The uncanny valley,” *Energy*, vol. 7, no. 4, pp. 33–35, 1970.
- [17] X. Xu, X. Cao, and Q. Wu, “Impact of educational agents on students’ learning outcomes: A meta-analysis,” *Frontiers in Psychology*, vol. 17, p. 1707196, 2026.
- [18] H. Li, Z. Wang, L. Ding, J. Zhang, and G. Wang, “The facts about the effects of pedagogical agents on learners’ cognitive load: A meta-analysis based on 24 studies,” *Frontiers in Psychology*, vol. 16, p. 1635465, 2025.
- [19] J. Dang, K. M. King, and M. Inzlicht, “Why are self-report and behavioral measures weakly correlated?” *Trends in Cognitive Sciences*, vol. 24, no. 4, pp. 267–269, 2020.
- [20] L. S. Vygotsky, *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, 1978.
- [21] C. Bartneck, D. Kulić, E. Croft, and S. Zoghbi, “Measurement instruments for the anthropomorphism, animacy, likeability, perceived intelligence, and perceived safety of robots,” *International Journal of Social Robotics*, vol. 1, no. 1, pp. 71–81, 2009.
- [22] J. Brooke, “Sus: A ‘quick and dirty’ usability scale,” in *Usability Evaluation in Industry*. Taylor & Francis, 1996, pp. 189–194.
- [23] S. G. Hart and L. E. Staveland, “Development of nasa-tlx (task load index): Results of empirical and theoretical research,” *Advances in Psychology*, vol. 52, pp. 139–183, 1988.
- [24] E. R. Thompson, “Development and validation of an internationally reliable short-form of the positive and negative affect schedule (panas),” *Journal of Cross-Cultural Psychology*, vol. 38, no. 2, pp. 227–242, 2007.